

Weather App File Guide

A simple explanation of what each project file does

Project: Tropical Cyclone Information System

Prepared September 9, 2026

How the app works

The project has a public website, an admin portal, and a MySQL database. The public pages display information. Admin pages update information. PHP API files connect the browser to the database.

```
Admin edits storm data
|
v
MySQL database: cyclone_db
|
v
PHP API returns JSON
|
v
JavaScript reads the JSON
|
v
Public pages show the information
```

Filename note: There is no `lunch.json`. The file is `.vscode/launch.json`. It tells VS Code how to open the site in Chrome.

Root and VS Code files

`index.html`

The homepage. It contains the video hero, storm statistics, upcoming storm area, education links, and resource previews.

`composer.json`

Lists the PHP package required by the app. The project uses PHPMailer to send password-reset emails.

`composer.lock`

Records the exact versions of the installed PHP packages so the project uses predictable dependencies.

`.gitignore`

Tells Git which files should not be saved in the repository, such as temporary files or private configuration.

.vscode/launch.json

Defines a Chrome debugging setup for `http://localhost:8080`, using the project folder as the web root.

Admin folder

These PHP files are used by administrators to log in and manage cyclone records.

admin/auth.php

Checks whether an admin is logged in. If not, it redirects the visitor to `login.php`. This protects admin-only pages.

admin/login.php

Shows the login form and checks the username and password against the `admins` database table. A successful login creates a PHP session.

admin/logout.php

Clears the admin session, removes the session cookie, and sends the user back to the login page.

admin/forgot-password.php

Starts password recovery. It creates a six-digit code, saves it in the database, and emails it using PHPMailer.

admin/reset-password.php

Checks the six-digit code. If it is valid and not expired, it hashes and saves the new password.

admin/dashboard.php

Shows an admin overview: total cyclones, latest year, strongest storm, recent records, and the current upcoming storm.

admin/edit-storm.php

Lets the admin add or edit the current upcoming storm. The saved values later appear on the public pages.

admin/cyclones.php

Shows historical cyclones in a searchable, filterable, paginated table.

admin/cyclone-form.php

Adds a new cyclone or edits an existing one. It validates names, dates, wind speeds, categories, and warning signals before saving.

admin/cyclone-delete.php

Deletes a historical cyclone. It only accepts a POST request, which helps prevent accidental deletion from a normal link.

admin/helpers.php

Contains shared admin functions for database connections, form values, error messages, CSS error classes, and dropdown options.

admin/nav.php

Displays the admin navigation bar and highlights the current admin section.

admin/partials/site-footer.php

Displays the footer used by admin pages.

Security reminder: The email account password is currently written directly in `admin/forgot-password.php`. It should be moved to a protected environment variable before deployment.

API folder

api/get_cyclones.php

Reads all historical cyclone rows from MySQL and returns them as JSON. Public JavaScript uses this for tables, statistics, charts, and comparisons.

api/get_upcoming_storm.php

Reads the active upcoming storm and returns it as JSON. If no storm is active, the public site displays an empty state.

Public HTML pages

pages/about.html

Explains the platform's purpose, audience, goals, and official data source.

pages/compare-storms.html

Lets users compare an upcoming storm with historical cyclones or another selected storm.

pages/contact.html

Shows contact information, emergency guidance, official sources, and a browser-only feedback form.

pages/historical-data.html

Shows historical cyclone records with searching, filters, pagination, statistics, and CSV export.

pages/resources.html

Lists emergency hotlines, government agencies, disaster resources, and useful external websites.

pages/education/index.html

The Education Hub. It contains topic cards, resource tiles, preparedness facts, and links to detailed guides.

pages/education/warning-signals.html

Explains PAGASA warning signals, wind ranges, lead times, and recommended actions.

pages/education/evacuation-guide.html

Explains when and how to evacuate and includes a go-bag checklist.

pages/education/preparedness-tips.html

Explains what to do before, during, and after a typhoon and includes an emergency-kit checklist.

Reusable partials

partials/header.html

A reusable public header containing the announcement bar, logo, navigation, Education menu, About and Contact links, and admin login link.

partials/footer.html

A reusable public footer containing site information and navigation links.

Several pages currently contain equivalent header and footer markup directly instead of loading these partial files dynamically.

JavaScript files

js/main.js

Shared browser behavior: mobile menu, Education dropdown, announcement dismissal, header effects, active links, and reveal animations.

js/home-overview.js

Loads cyclone data and calculates homepage totals, year range, average, strongest storm, busiest year, and newest cyclone.

js/upcoming-storm.js

Loads the active storm from the API and places its values into the public page.

js/storm-category.js

Converts sustained wind speed into a PAGASA-style category such as Tropical Depression, Typhoon, or Super Typhoon.

js/topic-models.js

Stores the education topics, titles, icons, summaries, and detailed text.

js/topic-detail-modal.js

Builds and controls the Education Hub popup. It supports closing, Escape, focus restoration, and keyboard navigation.

js/pages/compare-storms.js

Loads storms, finds similar storms, renders storm selectors, compares values, and displays winner or tie results.

js/pages/historical-data.js

Loads records, applies filters, creates the table and pagination, calculates statistics, and exports filtered CSV data.

js/pages/evacuation-guide.js

Creates the evacuation checklist, saves checked items in the browser, updates progress, and supports reset controls.

js/pages/preparedness-tips.js

Creates the preparedness checklist, saves its state in the browser, updates progress, and supports reset controls.

CSS files

css/base.css

Global styles: reset rules, colors, typography, spacing, focus styles, layout helpers, and animations.

css/admin.css

Styles the admin login, dashboard, forms, tables, alerts, cards, status colors, and admin navigation.

css/components/header.css

Styles the announcement bar, public header, logo, navigation, dropdown, login button, and mobile menu.

css/components/buttons.css

Provides shared public button styles.

css/components/footer.css

Styles the public and admin footer.

css/pages/home.css

Styles the homepage hero video, feature cards, overview cards, and storm profile.

css/pages/about.css

Styles the About page sections, cards, links, and call-to-action area.

css/pages/compare-storms.css

Styles storm selectors, comparison tables, result panels, controls, and dialogs.

css/pages/contact.css

Styles the contact hero, feedback form, emergency note, and official-source panel.

css/pages/education.css

Styles the Education Hub, topic cards, resource tiles, fact panel, and topic modal.

css/pages/evacuation-guide.css

Styles evacuation decisions, reminders, tables, checklists, and progress components.

css/pages/historical-data.css

Styles the historical-data filters, statistics, table, pagination, search, and CSV button.

css/pages/preparedness-tips.css

Styles the preparedness phases, emergency-kit checklist, tips, and call-to-action.

css/pages/resources.css

Styles hotline cards, agency cards, resource links, and resource-page animations.

css/pages/warning-signals.css

Styles warning-signal cards, reference tables, reminders, and responsive layouts.

Assets

assets/images/Logo.png

The main logo used by the public header and admin navigation.

assets/images/Logo_original.png and Main_Logo.png

Other logo versions kept as source or backup images.

assets/images/Storm_bg.jpg

Storm photograph used for hero backgrounds and as the homepage video fallback image.

assets/images/bg.png, bg1.jpg, bg2.jpg, bg3.jpg, bg4.jpg

General and education-related background or thumbnail images.

assets/images/education-hub-background-v3-color.png

Education Hub background.

assets/images/evacuation-guide-background-color.png

Evacuation Guide background.

assets/images/preparedness-tips-background-color.png

Preparedness Tips background.

assets/images/warning-signals-background-color.png

Warning Signals background.

assets/images/education-guide-thumbnail.png, how-storms-work-thumbnail.png, contact-resources-dialing-thumbnail.png

Homepage thumbnails for education, storm science, and contact/resource sections.

assets/video/background_video.mp4

Muted looping video used behind the homepage hero section.

Scripts folder

scripts/__seed_admin.php

Temporary helper that creates a test admin account for automated testing.

scripts/check-logo-alpha.js

Checks whether logo pixels are transparent.

scripts/make-logo-transparent.js

Removes a near-white logo background and creates a transparent image.

scripts/tmp_add_reveal.js

Temporary script that adds reveal-animation attributes to HTML.

scripts/tmp_fix_reveal.js

Temporary script that repairs reveal-animation attributes.

scripts/tmp_data_scan.ps1

PowerShell research tool for scanning storm-data files, finding duplicates, and reading metadata.

scripts/verify-best-match.tmp.js

Temporary test for the best historical-storm match calculation.

scripts/verify-compare-panels.tmp.js

Temporary test for comparison selectors, panels, winner states, and reset behavior.

scripts/verify-topic-modal.tmp.js

Temporary test for the Education topic popup, closing behavior, focus, and keyboard controls.

Vendor folder

vendor/autoload.php

Automatically loads installed PHP packages so the application can use PHPMailer.

vendor/phpmailer/

Third-party PHPMailer code used to send password-reset emails.

vendor/composer/

Composer-generated files that help PHP locate and load installed packages.

Vendor files normally should not be edited manually because they are generated or supplied by external packages.

Example: updating the upcoming storm

When an administrator edits the storm:

1. `admin/edit-storm.php` validates and saves the information.
2. The information is stored in the MySQL `upcoming_storm` table.
3. `api/get_upcoming_storm.php` reads the active row and returns JSON.
4. `js/upcoming-storm.js` receives the JSON in the browser.
5. The homepage and comparison page display the new storm information.